

Informe de Capacidad y Rendimiento de Infraestructura

Proyecto: Plataforma de Gestión de Bancas de Lotería

Fecha: 26 de junio de 2026

Destinatario: Cliente / Administración

1. Resumen Ejecutivo

Este informe detalla las capacidades de procesamiento, volumen de transacciones y concurrencia de usuarios que soporta la infraestructura actual de la plataforma. El sistema ha sido diseñado bajo una arquitectura de alta disponibilidad y resiliencia, utilizando tecnologías líderes en la industria como Render Standard, Upstash Redis y Supabase Pro (PostgreSQL) con multiplexación de conexiones.

La plataforma está optimizada para manejar de manera segura el comportamiento de ráfaga típico de los sorteos de lotería (donde el 90% del tráfico se concentra en los 10 minutos previos al cierre).

2. Componentes de la Infraestructura

La arquitectura distribuye la carga de trabajo de manera eficiente entre diferentes capas especializadas:

graph TD

A["Vendedores (App Móvil Kotlin Nativa)"] -->|APIs REST ligeras| B["Render Web Service (Standard 2GB)"]

B -->|Caché Ultra-Rápida L1/L2| C["Upstash Redis"]

B -->|Conexiones Multiplexadas| D["Supervisor (Pooler)"]

D -->|Persistencia y Transacciones| E["Supabase DB (Pro + Small Compute)"]

- Servidor de Aplicación (Render Standard - 2GB RAM / 1 vCPU): El backend corre directamente sobre Render bajo el dominio `backend-bancas-yk1i.onrender.com`. Cuenta con auto-escalado dinámico a 2 instancias cuando el uso de CPU o memoria supera el 60%.
- Base de Datos (Supabase Pro + Small Compute - 2GB RAM / 2 vCPU): Utiliza el pooler Supervisor (puerto 6543) para gestionar de forma eficiente miles de conexiones concurrentes sin saturar el servidor físico.
- Capa de Aceleración (Upstash Redis + Memoria L1): Resuelve consultas estáticas y repetitivas (como reglas de bloqueo y horas límite) en menos de 10 milisegundos.

3. Impacto de la Aplicación Móvil Nativa (Kotlin APK)

El uso de una aplicación móvil nativa en lugar de una aplicación web móvil convencional reduce drásticamente el consumo de recursos en el servidor, incrementando el límite real de usuarios concurrentes:

1. Tráfico de Red Minimalista (REST API pura): El servidor de Render no necesita servir archivos estáticos pesados (HTML, paquetes grandes de JavaScript de React/Next.js o archivos de diseño CSS). El servidor de Render solo procesa y responde con JSON ligeros (típicamente de menos de 1 KB).
2. Cero Renderizado en Servidor: Todo el renderizado visual de la interfaz se ejecuta de forma local en el dispositivo del vendedor mediante Android Jetpack Compose, consumiendo la CPU del teléfono y dejando la CPU del servidor de Render libre para atender transacciones.
3. Gestión de Conexiones Optimizada (OkHttp): La app nativa utiliza la biblioteca OkHttp, la cual mantiene pools de conexiones HTTP abiertos y reutilizables hacia Render (`backend-bancas-yk1i.onrender.com`) de forma inteligente, evitando el protocolo de "handshake" de red en cada venta de ticket.

4. Métricas de Capacidad Estimadas

Las métricas se calculan bajo condiciones extremas y se dividen según el tipo de actividad realizada por los vendedores:

4.1. Registro de Ventas (Operaciones de Escritura)

El registro de ventas requiere validación transaccional estricta en tiempo real para evitar la sobreventa (oversell) de números.

- Capacidad Base (1 Instancia activa):
- ~39 ventas por segundo (2,340 ventas por minuto).
- Capacidad en Pico de Carga (2 Instancias activas mediante auto-escalado):
- ~78 ventas por segundo (4,680 ventas por minuto).

4.2. Consultas y Lecturas de Datos

- Consultas de Operación Rápida *(ej: ver bloqueos de números, verificar estado de sorteos, home de la app):*
- Resueltas por la caché L1 (en memoria del servidor) y L2 (Redis).
- Capacidad: ~1,000 a 2,000 consultas por segundo (60,000 a 120,000 por minuto).
- Consultas Administrativas Directas *(ej: ver históricos de tickets, estados de cuenta, reportes de cierre):*
- Resueltas directamente por la base de datos de manera optimizada.
- Capacidad: ~150 a 250 consultas por segundo (9,000 a 15,000 por minuto).

4.3. Vendedores Simultáneos (Usuarios Concurrentes)

- Escenario de Operación Normal (Día a Día):
- Los vendedores ingresan a la app, consultan saldos y venden de forma espaciada (promedio de 1 ticket cada 20 a 30 segundos).

- Capacidad recomendada: 2,000+ vendedores activos simultáneamente.
- Escenario Frenético (Minutos previos al cierre del sorteo):
 - Todos los vendedores intentan registrar ventas de forma masiva y consecutiva (promedio de 1 ticket cada 5 segundos).
 - Capacidad recomendada (sin retrasos): ~400 vendedores operando en ráfaga simultánea.
 - *Nota: Si este número se supera temporalmente, el sistema no se cae; las peticiones hacen fila de forma ordenada, incrementando levemente el tiempo de respuesta del botón "Vender" de 0.6s a unos pocos segundos.*

5. Garantías de Estabilidad y Control de Riesgos

La plataforma cuenta con mecanismos de defensa automatizados para proteger la continuidad del negocio:

1. Protección contra duplicados (Idempotencia): El sistema utiliza bloqueos atómicos en Redis. Si un vendedor presiona el botón de venta dos veces por error, el sistema detecta la duplicación en milisegundos y procesa solo una transacción, previniendo errores contables.
2. Circuit Breaker en Redis: Si la base de datos de Redis alcanza su límite de cuota o se desconecta temporalmente, la aplicación no falla. El backend "abre el circuito" y redirige el tráfico de forma segura directamente a la base de datos PostgreSQL de Supabase.
3. Coalescencia de Peticiones en Vuelo (In-Flight Request Coalescing): Si 100 vendedores solicitan las mismas reglas de sorteos al mismo tiempo, el servidor ejecuta una única consulta a la base de datos y distribuye la respuesta a los 100 usuarios, disminuyendo el esfuerzo del servidor en un 99%.

6. Conclusión

La combinación de una App Móvil Nativa de alto desempeño junto con una infraestructura auto-escalable y optimizada ofrece una base sumamente sólida. Su capacidad para soportar hasta 400 vendedores en venta frenética simultánea y más de 2,000 vendedores en uso normal cubre con holgura la operación proyectada.

En caso de que el negocio crezca más allá de estos umbrales, la arquitectura permite realizar un escalado horizontal (aumentar réplicas en Render) o vertical (aumentar capacidad en Supabase) con cero minutos de interrupción del servicio.